Activity 08 Verify withdrawal controls against reentrancy

Checks-effects-interactions plus entered guard prevent recursive withdrawal; an actual adversarial contract probes the boundary.

Mapped learning evidence: A4, A5, K4. This folder is self-contained; copy it without any other activity. Contract.sol is editable source; contract.json is a reproducible compiled training artifact. Mock records are synthetic.

Business acceptance

Record the actor, permitted function, expected state and rejection condition before running code. Map the Solidity requirement to each assertion in run_activity.py. Evidence consists of actual EVM state, receipt status, event logs and gas; Python is the client and tester.

Detailed procedure

1. Install Python 3.12 or 3.13 and Node.js 22 LTS. Open a terminal in this activity folder. Confirm python3 --version and node --version. Use a fresh directory and an isolated training environment.

2. Read mock-data.json and mock-data.csv. Identify the record ID, integer quantity and evidence boundary. No private record or production signing key is required.

3. Open prompts.md. Submit the generation prompt to your approved coding assistant. Save the generated draft separately and compare it to the supplied runnable baseline. Ask the review prompt to locate authorization and state controls.

4. Create a Python environment with python3 -m venv .venv. Activate it using source .venv/bin/activate on macOS/Linux or .venv\Scripts\activate on Windows. Run python -m pip install -r requirements.txt.

5. Run npm install (then npm ci on subsequent runs after package-lock.json exists). The compiler is pinned to 0.8.30 and OpenZeppelin to 5.4.0; this reproducible exercise is not a recommendation to ignore future security releases.

6. Read Contract.sol. Mark each external/public state-changing function and identify who may call it. Trace SafeVault against the acceptance requirement above. For ERC contracts, inspect the imported OpenZeppelin base, rather than treating an abbreviated example as the standard implementation.

7. Run node compile.cjs. Confirm the compiled contract name and version. Inspect contract.json for ABI and nonempty bytecode. Fix compiler errors at the indicated file and line; never suppress errors. The target EVM is Cancun with optimizer 200 runs.

8. Run python run_activity.py. This creates a fresh real Py-EVM chain with funded synthetic accounts, deploys bytecode, executes transactions, checks state and negative cases, and writes results.json and deployment.json. It does not mock a blockchain.

9. Inspect results.json. Require assertions=passed and receipt gasUsed/block/hash fields. Read each assertion in the script and explain the business risk if it were removed. Inspect deployment.json and reconcile the contract address and chain ID.

10. Change one mock record, recompile if contract source changed and rerun. Explain why the digest changes for changed evidence and why each fresh Py-EVM run has independent state. Do not treat a test chain transaction as an external deployment.

11. Submit Contract.sol, your requirement-to-assertion matrix, prompts used, results.json, deployment.json and any relevant event/gas outputs. State which negative cases were executed and the remaining boundary.

Local RPC extension

To share a persistent chain with a browser wallet, launch an isolated Anvil development chain using anvil --chain-id 31337 --host 127.0.0.1 --port 8545. Keep the process running and use only its disposable default test accounts. Run python run_activity.py --rpc http://127.0.0.1:8545. The RPC path requires at least three unlocked local test accounts and uses node-managed test signing. Never point this script at a production RPC. Save the manifest generated by that run; the default Py-EVM chain is in-process and cannot be reached by MetaMask.

Verify and diagnose

- All positive assertions must pass and every invalid/unauthorized call must be rejected. A transaction hash alone is insufficient.
- If imports fail, activate the activity .venv and reinstall requirements; use python -m pip so pip and python refer to the same interpreter.
- If Solidity imports fail, run npm ci in this folder and inspect package-lock.json.
- If a transaction reverts, inspect sender, value in wei and current state. Read the require condition before changing code.
- If RPC connection fails, confirm Anvil is still running and the URL/port matches; the browser and script must use the same chain ID and manifest.
- Gas varies with EVM/compiler/state. Compare receipt gasUsed only with the same locked toolchain and test state.
- Reset by restarting the local chain or rerunning the default Py-EVM script. Preserve results before reset.

Reflection and clean up

Explain the off-chain/on-chain boundary, a failing requirement, the assertion that catches it, and the artifact another developer needs to reproduce your work. Stop Anvil when finished. Remove the local .venv/node_modules if disk space is needed; keep source, locks, mocks and evidence.

Adversarial receive callback exact execution sequence

Depositor account0 deposits1ETH. Deploy ReenterProbe with vault address. Account1 funds attack0.1ETH; receive callback attempts vault.withdraw recursively and records blocked=true. Read probe balance entry0 and vault balance1ETH. Withdraw original depositor; vault ends zero.

Expected observable state: ReenterProbe.blocked=true; victim deposit preserved1ETH; no duplicate payout.

Edit and verify task

Temporarily remove guard and zeroing in a separate vulnerable copy, demonstrate theft only in this isolated chain, then restore controls. Add your new assertion before modifying source. Submit the original failing output and corrected passing output with a source diff.